

# LeRobot ACT Policy Generalization on CALVIN ABC→D

Assignment Report

GitHub Repository: **To be replaced after public GitHub upload**

Model Weights: **To be replaced after model-weight upload**

June 26, 2026

## Abstract

This report studies cross-environment generalization for Action Chunking with Transformers (ACT) using the LeRobot policy stack on the CALVIN ABC→D benchmark. Two policies are trained with the same ACT architecture and hyperparameters: an A-only policy trained only on environment A and an ABC-joint policy trained on mixed A, B, and C data. Both are evaluated zero-shot in unseen environment D using offline action L1 error and closed-loop CALVIN rollouts. The final implementation uses two RGB camera views, robot state, official CALVIN language embeddings, action chunk size 10, temporal ensembling, and an effective stop at 88k training steps. The A-only model obtains 0.136686 D offline action L1, 22% D SR@1, and 0.32 average completed subtasks. The ABC-joint model obtains 0.114896 D offline action L1, 38% D SR@1, and 0.56 average completed subtasks. The main finding is narrow but consistent: under the same ACT design, mixing A/B/C training data improves zero-shot transfer to the held-out visual distribution D.

## 1 Introduction

Robotic imitation learning is easy to evaluate on the scene where demonstrations were collected, but much harder to trust after the visual environment changes. Background color, camera viewpoint, object placement, and small appearance details can change the visual features used by a policy even when the intended manipulation skill is the same. This issue is central to embodied AI: a robot policy should learn a task-relevant mapping from observation and goal to action, rather than memorizing one tabletop scene.

The assignment asks for a compact ACT-based study on CALVIN [1]. CALVIN is a language-conditioned long-horizon manipulation benchmark with several related but visually distinct simulated environments. In the ABC→D protocol, environments A, B, and C are available for training, while D is held out for zero-shot evaluation. This report follows that protocol with two final formal policies. The first policy is trained only on A. The second policy is trained on A, B, and C jointly. The architecture, optimizer, loss, input resolution, action chunk length, language-conditioning mechanism, evaluation scripts, and stopping rule are fixed between the two runs.

ACT, introduced by Zhao et al. [2], predicts a short chunk of future actions instead of a single action. This design reduces the effective decision horizon and can make fine manipulation smoother. The same property

also creates a failure mode under visual shift: if the first observation is interpreted incorrectly, a whole chunk can be biased in the wrong direction. The experiment is therefore designed around one practical question: does the broader visual support of A/B/C training make ACT more robust when it is rolled out in unseen D?

The answer in this workspace is yes, with a clear but not exaggerated margin. A-only fits its own training environment better, reaching lower in-domain validation L1. However, ABC-joint obtains lower held-out D action error and higher closed-loop D success. This distinction is important for reporting: training-validation loss alone is not a sufficient proxy for cross-environment deployment quality.

## 2 Related Work

**Language-conditioned manipulation benchmarks.** CALVIN, short for Composing Actions from Language and Vision, was proposed as a benchmark for long-horizon language-conditioned robot manipulation [1]. Its goal is to evaluate policies that map visual observations and natural-language goals to long sequences of actions. The benchmark emphasizes zero-shot generalization to novel instructions and novel environments. The official CALVIN repository and dataset release provide the simulation environment, dataset splits, and task oracle used by this report [5].

**Action chunking for imitation learning.** ACT was introduced in the ALOHA work on low-cost bi-manual manipulation [2]. Its key idea is to predict action chunks with a Transformer policy and a variational training objective. In the original setting, chunking helped reduce compounding error by shortening the effective horizon of autoregressive control. LeRobot exposes ACT as a lightweight policy for imitation learning and recommends it as a practical starting point because it trains quickly and has modest compute requirements [3]. This assignment uses that LeRobot ACT implementation rather than replacing it with a larger policy class.

**LeRobot tooling and dataset format.** LeRobot is an open-source robotics learning stack from Hugging Face that provides policy implementations, dataset tooling, and training/evaluation utilities [4, 3]. The local data used here is a parquet conversion of CALVIN into a LeRobot-compatible format [6]. The conversion is useful because it lets the experiment reuse existing Python, PyArrow, and LeRobot-style loading tools while preserving the CALVIN ABC→D split semantics.

## 3 Dataset and Environment

### 3.1 Split Definition

The final protocol uses the CALVIN ABC→D split. Training data comes from environments A, B, and C. Zero-shot evaluation uses environment D only. The A-only policy sees only A during training and training-validation. The ABC-joint policy sees a mixture of A, B, and C during training and training-validation. D is not used for model selection during training; it is evaluated after the best 88k checkpoint is selected from the training-environment validation metric.

**Table 1:** Local CALVIN episode coverage used by the final protocol.

| Environment | Role                       | Episodes |
|-------------|----------------------------|----------|
| A           | A-only and ABC-joint train | 6089     |
| B           | ABC-joint train            | 6115     |
| C           | ABC-joint train            | 5666     |
| D           | zero-shot evaluation       | 1087     |

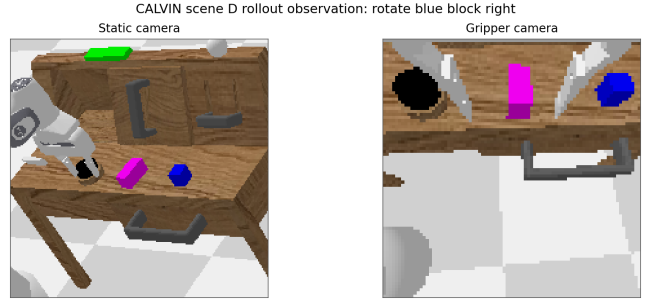
### 3.2 Observation and Action Contract

The policy input contains two RGB views and a state vector. The static camera captures the tabletop scene; the gripper camera captures wrist-centric visual detail. Robot proprioception contributes 15 state dimensions. The language goal is represented by the official 384D CALVIN SBERT embedding associated with the task annotation. The embedding is concatenated with proprioception, so the effective state dimension is 399.

The action target is the 7D CALVIN relative action: six dimensions for end-effector motion and one gripper command. A training example at time  $t$  supervises a future action chunk  $a_{t:t+K-1}$ . When a chunk reaches beyond the end of an episode, padding masks remove invalid positions from the reconstruction loss. This is important because otherwise short terminal windows would bias the loss toward padded values.

### 3.3 Closed-loop Runtime

Closed-loop evaluation uses the official CALVIN PyBullet environment and task oracle. The rollout script loads a LeRobot checkpoint, converts CALVIN observations into the policy input format, decodes an action, and sends that action back to the environment. No scripted planner, oracle executor, or replay fallback is used to execute robot actions. The metric therefore reflects the learned ACT policy path.



**Figure 1:** Example observation from the official CALVIN scene-D rollout environment used for zero-shot testing. The left panel shows the static camera and the right panel shows the gripper camera for the task annotation “take the blue block and rotate it to the right.”

## 4 Method

### 4.1 ACT Objective

ACT predicts a block of future actions from the current observation:

$$\hat{a}_{t:t+K-1} = \pi_{\theta}(o_t, z),$$

where  $z$  is the latent variable used by the ACT variational objective. During training, the encoder observes the target action sequence and infers a posterior over  $z$ . The decoder reconstructs the action chunk conditioned on image features, state features, and the latent variable. The objective is:

$$\mathcal{L} = \mathcal{L}_{L1} + \lambda_{KL}\mathcal{L}_{KL}, \quad (1)$$

where  $\mathcal{L}_{L1}$  is the masked action reconstruction loss and  $\mathcal{L}_{KL}$  regularizes the latent posterior toward a unit Gaussian. The final runs use  $\lambda_{KL} = 10$ .

### 4.2 Language-conditioned State Construction

CALVIN tasks are goal-conditioned. A visually similar initial state can require different next actions depending on whether the instruction is, for example, to move a slider, open a drawer, or switch on a light. The final implementation therefore adds language conditioning while keeping the ACT network simple. The official CALVIN SBERT embedding is appended to the robot state before it is passed to LeRobot ACT. This is a small architectural change: the action decoder, chunking mechanism, optimizer, and loss remain ACT.

### 4.3 Chunk Length and Temporal Ensemble

The final chunk size is 10. A longer chunk can produce smoother behavior, but it can also carry a wrong visual interpretation for too many control steps. A short chunk increases the frequency at which the policy can correct from a new observation. Temporal ensembling is used with coefficient 0.01 to combine overlapping chunk predictions. In this report it is treated as an inference stabilizer, not as a separate controller.

## 5 Experimental Protocol

### 5.1 Training Runs

Both formal runs were trained to an effective stop at step 88000. The command cap was 100000 steps, but training was intentionally stopped after the step-88000 validation pass and the best checkpoint at step 88000 was used for evaluation. Both policies use the same architecture and hyperparameters; only the training environment list differs.

### 5.2 Offline Evaluation

Offline evaluation computes action reconstruction loss on the held-out D validation split. It does not interact with the simulator, so it is not a success-rate metric. Its value is still useful because it checks whether the policy predicts actions closer to D demonstrations when conditioned on D observations and language goals.

### 5.3 Closed-loop Evaluation

Closed-loop evaluation runs 50 CALVIN task chains in scene D. The main metric in this report is SR@1, the fraction of chains in which the first subtask is completed. Average completed subtasks is also reported because CALVIN chains are long-horizon. The metric is stricter than offline loss because the model must recover from its own states after each action.

## 6 Results

### 6.1 Training Convergence

Figure 2 shows the final continuation segment. The A-only model reaches lower training-environment validation L1, 0.052623, because its validation distribution is narrower. The ABC-joint model reaches 0.073414, which is higher in-domain but expected because it must fit a broader mixture of visual scenes. This difference is not a failure of the ABC run. It indicates that in-domain validation L1 and cross-environment deployment quality are not the same quantity.

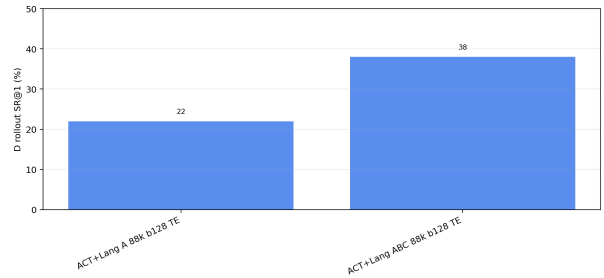
### 6.2 Zero-shot D Performance

**Table 3:** Final zero-shot D comparison. Both policies use the same ACT architecture and hyperparameters.

| Model         | Val L1   | D L1     | D SR@1 | Avg. len |
|---------------|----------|----------|--------|----------|
| A-only 88k    | 0.052623 | 0.136686 | 22%    | 0.32     |
| ABC-joint 88k | 0.073414 | 0.114896 | 38%    | 0.56     |

Table 3 is the main experimental evidence. A-only has better training-environment validation L1, but ABC-joint is better on both held-out D metrics. The D offline action L1 decreases from 0.136686 to 0.114896. Closed-loop SR@1 increases from 22% to 38%. The average completed subtasks increases from 0.32 to 0.56. These

three movements point in the same direction: the multi-environment training set improves zero-shot transfer.



**Figure 3:** Closed-loop SR@1 on unseen CALVIN D. The comparison is intentionally limited to the two final formal ACT policies.

### 6.3 Reading the Metric Gap

The most useful detail is not just that ABC-joint is higher on SR@1. It is that the relative ordering flips between in-domain validation and D evaluation. If the report only used training-validation L1, A-only would look better. Once the held-out D environment is used, ABC-joint is the stronger policy. That is exactly the generalization behavior the assignment is designed to probe.

## 7 Analysis

### 7.1 Why Multi-environment Training Helps

The A-only policy can rely on stable visual features of one scene. This makes training easier but also creates a risk: some features that are predictive in A may be incidental under D. ABC-joint training exposes the encoder to more backgrounds, object appearances, and layouts. The policy is therefore pressured to represent task-relevant local geometry, gripper-object relations, and language-conditioned intent rather than a single scene’s texture statistics. The held-out D results are consistent with this interpretation.

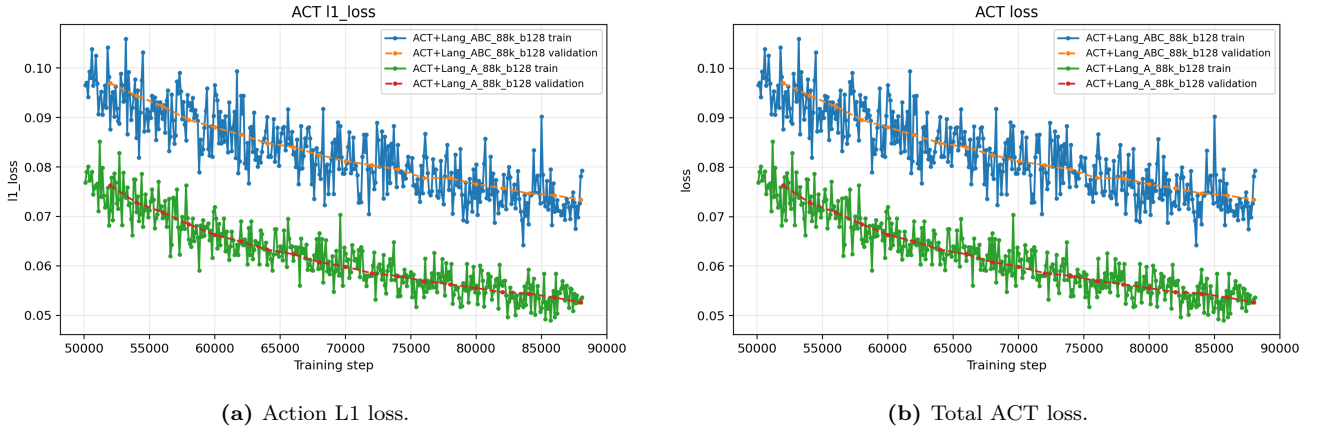
### 7.2 Action Chunking Under Distribution Shift

Action chunking helps when the observation is understood correctly. A single observation can generate a coherent short action segment, which reduces jitter and can stabilize contact timing. Under visual distribution shift, however, chunking has a second edge: a wrong interpretation can be propagated for several control steps. The chunk size of 10 is a practical compromise. It is long enough to preserve short-horizon smoothness but short enough to let new observations correct the control stream frequently.

Temporal ensembling works in the same spirit. It blends overlapping predictions to reduce abrupt action changes. In this experiment it is not credited as a separate controller, because it does not decide the task

**Table 2:** Shared hyperparameters and implementation settings for the final ACT policies.

| Category     | Setting                  | Value             | Notes                           |
|--------------|--------------------------|-------------------|---------------------------------|
| Data         | Format                   | HF parquet        | LeRobot-style CALVIN conversion |
| Data         | Train environments       | A or A,B,C        | only difference between runs    |
| Input        | Camera views             | static, gripper   | RGB images resized to 224       |
| Input        | State dimension          | 399               | 15 robot + 384 language         |
| Action       | Action dimension         | 7                 | relative pose + gripper         |
| ACT          | Chunk size               | 10                | masked near episode end         |
| ACT          | Transformer width        | 512               | shared across runs              |
| ACT          | Attention heads          | 8                 | shared across runs              |
| ACT          | Encoder / decoder layers | 4 / 1             | LeRobot ACTPolicy               |
| ACT          | Feed-forward dimension   | 3200              | shared across runs              |
| Optimization | Optimizer                | AdamW             | weight decay 1e-4               |
| Optimization | Learning rate            | 1e-5              | same for both runs              |
| Optimization | Batch size               | 128               | final continuation runs         |
| Loss         | Objective                | Action L1 + KL    | KL weight 10                    |
| Logging      | Tracking                 | W&B offline + CSV | curves exported locally         |
| Stopping     | Effective step           | 88000             | best checkpoint at step 88000   |

**Figure 2:** Training curves exported from W&B-compatible offline logs and local `metrics.csv` files. Solid lines are training metrics and dashed lines are validation metrics.

or plan the next symbolic action. It only smooths the learned ACT outputs.

### 7.3 Why Offline L1 Is Not Enough

Offline L1 compares predicted actions to demonstrations at fixed states. It is valuable for debugging because it isolates the model and preprocessing path from simulator dynamics. Closed-loop rollout is harder because the policy must live with the states it creates. A small action bias can change the next observation, which then changes the next action. This is why the report includes both metrics. The final claim is based on their agreement, not on one proxy metric alone.

### 7.4 Practical Failure Modes

The remaining errors are consistent with a lightweight behavior-cloning policy. Some failures are likely due to small object localization and gripper timing. Others are likely due to long-horizon drift after the first few actions. The final ACT policy does not include a learned high-level subtask planner, a large pretrained

vision-language backbone, or multi-seed model selection. Those choices keep the experiment close to the assignment requirement, but they also limit the absolute success rate.

## 8 Reproducibility

### 8.1 Artifacts

All final artifacts are kept in the repository. The key paths are:

- A-only checkpoint: `outputs/train/act_lang_a_only_100k_b128_v1/best`
- ABC checkpoint: `outputs/train/act_lang_abc_joint_100k_b128_v1/best`
- result summary: `reports / formal_results_summary.json`
- figures: `reports/figures/formal_protocol/`
- scene-D simulation snapshot: `reports/figures/formal_protocol/calvin_scene_d_rollout_snapshot.png`

- checksum file: `weights/SHA256SUMS.txt`
- upload archive: `dist/model_weights_act_lang_88k_b128_20260626.tar.gz`

## 8.2 Environment

The local environment used Python 3.13.5, LeRobot 0.5.2, PyTorch 2.10.0+cu128, torchvision 0.25.0+cu128, PyBullet 3.2.7, and W&B offline logging. The GitHub-facing README includes the environment file, data preparation commands, training commands, offline evaluation commands, rollout commands, and PDF build command.

## 8.3 Claim Boundaries

The report makes one supported claim: under the fixed final ACT protocol, ABC-joint training generalizes better to D than A-only training. It does not claim leaderboard-level performance. It also does not claim that 88k is a globally optimal stopping point. The evidence is single-seed but complete for the requested comparison: both final checkpoints were trained, both were evaluated offline on D, and both were rolled out in D using the same runtime path.

## 9 Conclusion

This project completes the required LeRobot ACT comparison on CALVIN ABC→D. With the same architecture and hyperparameters, the ABC-joint model outperforms the A-only model on the held-out D environment: D offline action L1 improves from 0.136686 to 0.114896, D SR@1 improves from 22% to 38%, and average completed subtasks improves from 0.32 to 0.56. The result shows that multi-environment ACT training improves cross-environment robustness under visual distribution shift, while the action chunking mechanism provides a useful but limited source of short-horizon stability.

## References

- [1] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022. doi: [10.1109/LRA.2022.3180108](https://doi.org/10.1109/LRA.2022.3180108); arXiv: [2112.03227](https://arxiv.org/abs/2112.03227).
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems XIX*, Daegu, Republic of Korea, July 2023. doi: [10.15607/RSS.2023.XIX.016](https://doi.org/10.15607/RSS.2023.XIX.016).
- [3] Hugging Face. LeRobot Documentation: ACT (Action Chunking with Transformers). <https://huggingface.co/docs/lerobot/en/act>, accessed June 26, 2026.

- [4] Hugging Face. LeRobot: Making AI for Robotics More Accessible with End-to-End Learning. <https://github.com/huggingface/lerobot>, accessed June 26, 2026.
- [5] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN Official Repository. <https://github.com/mees/calvin>, accessed June 26, 2026.
- [6] F. Wang. calvin-task-ABC-D-lerobot dataset. Hugging Face Datasets, <https://huggingface.co/datasets/fywang/calvin-task-ABC-D-lerobot>, accessed June 26, 2026.